Resumen SOLID
Nicolás Alejandro Diosa Benavides
Ingesoft II - Sergio Enrique Vargas Pedraza

Este tema de SOLID ya había sido tocado en la materia anterior Ingesoft I, y según lo que entendí es una serie de principios a seguir a la hora de generar clases o en general diseñar un programa.

Ahora si entrando en materia del video, se dice que son 5 principios, el número de letras por las siglas de la palabra SOLID, aquí voy a enumerar cada uno de ellos y dar una pequeña explicación lo más concisa posible de lo que entendí y lo que significan en palabras mías:

- **S**ingle responsibility principle: Un componente de software debe tener una única función (este principio me parece muy contextual).

  ¿CÓMO IDENTIFICAR SU INCUMPLIMIENTO?

  La manera más clara para mí es sencillamente examinando el nombre del componente y viendo si cumple únicamente con ese nombre, si hay algo más fuera de lugar o que no corresponda al nombre del componente, es decir, cuando esté haciendo más cosas de las que debe, es porque se está violando este principio.

  EJM

  En los ejemplos trataré de desarrollar un programa muy abstracto que ilustre la situación de la manera más clara posible lo haré en python porque de java no me acuerdo mucho y además python es mucho más fácil de recordar; el ejemplo que utilizaré será sobre una fábrica de automóviles, en donde se ensamblan las piezas, el chasis con la carrocería, y se pinta la carrocería.

```python
SOLID_EJM_NADB.py > ...
class fabrica_carros:
    def __init__(self):
        ensamblador_carroceria = ensamblador_carroceria()
        pintor_carroceria = pintor_carroceria()
        ensamblador_final = ensamblador_final()


class ensamblador_carroceria:
    def __init__(self):
        pass
    def ensamblar_carroceria(piezas_carroceria):
        pass

class pintor_carroceria:
    def __init__(self):
        pass
    def pintar_carroceria(carroceria):
        pass

class ensamblador_final:
    def __init__(self):
        pass
    def ensamblar_chasis_carroceria(chasis,carroceria):
        pass

def main():
    fabrica1 = fabrica_carros()

main()
```

Como se puede ver en ésta imagen, existen diferentes clases para cada uno de los componentes, incluso cuando existen 2 ensambladores, al hacer cosas diferentes tienen necesariamente que ir separados, a pesar de que no haya un código de lo que hace cada uno en sus métodos se entiende que se trata de cosas diferentes y harán **únicamente** lo que se requiera de cada uno.

- **O**pen/Closed principle: Este principio describe que básicamente la clase o componente de software haga lo que tiene que hacer a pesar de que se vaya a hacer una escalabilidad en el código sin tener que modificar el código del componente original.

¿CÓMO IDENTIFICAR SU INCUMPLIMIENTO?

Cuando para abordar un problema o un caso del problema hay que tocar algún if o switch, otra manera es viendo también si al generar variaciones en los casos que se atienden, necesariamente hay que modificar la clase porque no está haciendo lo que se supondría.

EJM

```
Welcome        SOLID_EJM_NADB.py ●
SOLID_EJM_NADB.py > fabrica_carros > __init__ > colores
1   class fabrica_carros:
2       def __init__(self,colores):
3           ensamblador_carroceria = ensamblador_carroceria()
4           pintor_carroceria = pintor_carroceria(colores)
5           ensamblador_final = ensamblador_final()
6
7
8   class ensamblador_carroceria:
9       def __init__(self):
10          pass
11      def ensamblar_carroceria(piezas_carroceria):
12          pass
13
14  class pintor_carroceria:
15      def __init__(self,colores):
16          self.colores = colores
17
18      def pintar_carroceria(self,carroceria, color):
19          try:
20              self.colores.index(color)
21          except ValueError:
22              print("Color "+color+" no disponible")
23
24  class ensamblador_final:
25      def __init__(self):
26          pass
27      def ensamblar_chasis_carroceria(chasis,carroceria):
28          pass
29
30  def main():
31      fabrica1 = fabrica_carros()
32
33  main()
```

Continuando con el ejemplo de la fábrica, la clase pintor_carrocería y en especial su método *pintar_carrocería* se vuelve completamente escalable ante la variable colores, puesto que a pesar de que hayan 5 o 1000 colores el método hace lo que tiene que

hacer sin necesidad de modificar su código, lo único que se modificaría sería la lista de colores, mas no el código de la clase.

- **L**iskov substitution principle: Según el video el principio tiene como objetivo que toda clase hija o clase que implemente una interfaz debe cumplir el contrato para que el código no se rompa en ningún momento.
  Este principio no lo entendía por completo así que investigué y acudí a mis apuntes del semestre pasado para entenderlo mejor, es en general el principio que dicta que toda clase hija al sustituir a su clase padre en toda parte del código no debe colapsar y funcionar correctamente como funcionaba también su padre.

  ¿CÓMO IDENTIFICAR SU INCUMPLIMIENTO?
  Siempre que dentro del código se rompa algo que se ha cambiado con el objetivo de un funcionamiento análogo, es decir, siempre que se sustituya la clase y se rompa el código.

  EJM - Este ejemplo se adjunta al siguiente de ISP

- **I**nterface segregation principle: Este principio se refiere a que únicamente se utilicen interfaces con los atributos y métodos que se vayan a usar, puesto que muchas veces existen interfaces demasiado amplias para el uso en específico que es necesario darles.

  ¿CÓMO IDENTIFICAR SU INCUMPLIMIENTO?
  Propiedades que no se utilizan o se ponen en null, también interfaces demasiado grandes.

SOLID_EJM_NADB.py > pintor_carroceria

```python
class fabrica_carros:
    def __init__(self,colores):
        ensamblador_carroceria = ensamblador_carroceria()
        pintor_carroceria = pintor_carroceria(colores)
        ensamblador_final = ensamblador_final()

class ensamblador:
    def ensamblar(piezas):
        pass

class ensamblador_carroceria(ensamblador):
    def __init__(self):
        pass
    def ensamblar(piezas_carroceria):
        pass

class pintor_carroceria:
    def __init__(self,colores):
        self.colores = colores

    def pintar_carroceria(self,carroceria, color):
        try:
            self.colores.index(color)
        except ValueError:
            print("Color "+color+" no disponible")

class ensamblador_final:
    def __init__(self,chasis,carroceria):
        self.piezas=chasis+carroceria
    def ensamblar(piezas):
        pass

def main():
    fabrica1 = fabrica_carros()

main()
```

Aquí se aplica LSP puesto que se cumple el contrato de que los ensambladores funcionen para cada caso, y también el principio de segregación de interfaces pues solo se utiliza el único método de la interfaz en ambos ensambladores.

- **D**ependency inversion principle: La inversión de dependencias dicta que una clase no debe depender de una implementación concreta sino de un abstracción (por ejemplo un contrato de interfaz).

A mi criterio este principio es definitivamente el más difícil de entender, con la explicación del video no me quedó claro así que vi el siguiente video:
https://www.youtube.com/watch?v=OqxpDAjBr8o
En donde se explica más a detalle este principio.

¿CÓMO IDENTIFICAR SU INCUMPLIMIENTO?

Yo creo que la manera más fácil de identificar que una clase de "alto nivel" está dependiendo de una implementación concreta (de manera parecida al principio abierto y cerrado), es viendo si en otros casos de uso o flujos posibles es necesario generar otra instancia concreta y no es posible utilizar el mismo código para solucionar varios casos.

EJM

Al ver el código que utilice para ejemplificar el LSP y el ISP creo que sin quererlo también apliqué el DIP puesto que el método ensamblar funciona para todo tipo de piezas pero su uso solo se define en los constructores de las funciones, de manera que a fin de cuentas la abstracción de ensamblar atiende los casos de ensamblar tanto piezas de la carrocería como del chasis con la misma carrocería.

Después de hacer un par de modificaciones y arreglos para que el código ejecutara sin errores, dado que lo fui haciendo poco a poco y sin probarlo, este es el resultado final que reúne todos los ejemplos de manera abstracta de los principios SOLID:

Python

```python
class fabrica_carros:
    def __init__(self,colores,chasis,carroceria):
        self.ensamblador_carroceria1 = ensamblador_carroceria()
        self.pintor_carroceria1 = pintor_carroceria(colores)
        self.ensamblador_final1 =
ensamblador_final(chasis,carroceria)



class ensamblador:
    def ensamblar(piezas):
        pass

class ensamblador_carroceria(ensamblador):
```

```
    def __init__(self):
        pass
    def ensamblar(piezas_carroceria):
        pass

class pintor_carroceria:
    def __init__(self,colores):
        self.colores = colores

    def pintar_carroceria(self,carroceria, color):
        try:
            self.colores.index(color)
        except ValueError:
            print("Color "+color+" no disponible")

class ensamblador_final:
    def __init__(self,chasis,carroceria):
        self.piezas=chasis+carroceria
        print(self.piezas)
    def ensamblar(self):
        print(self.piezas)

def main():
    colores = ["amarillo","verde","azul"]
    fabrica1 = fabrica_carros(colores,"ev8","ec9")
    fabrica1.ensamblador_final1.ensamblar()



main()
```

REFLEXIÓN FINAL DEL VIDEO

Al final el creador del video sugiere que no siempre en proyectos de la industria y en especial en proyectos legacy que llevan mucho tiempo andando, se implementan los principios SOLID, o también se implementan parcialmente; por lo que es necesario tener en cuenta que

no están escritos en piedra y que si se puede mejorar el código es bueno mejorarlo pero si no es posible no se debe entrar en “crisis” en cambio tratar de mejorarlo al tiempo que sea posible y que sigan funcionando las cosas.